C Experimental Details

In this section, we provide details of the function vector extraction process (section 2.3), and the evaluation of function vectors (section 3).

Function Vector Extraction. We compute a function vector as the sum over the average output of several attention heads, where the average is conditioned on prompts taken from a particular task. We write this as $v_t = \sum_{a_{\ell j} \in \mathcal{A}} \bar{a}^t_{\ell j}$. How many attention heads should we use? To extract a function vector (FV), we first compute the task-conditioned mean activation of each head $\bar{a}^t_{\ell j}$, using $|P_t| = 100$ clean (uncorrupted) 10-shot prompts. We use this to identify a set of causal attention heads $\mathcal{A}$, which are ranked based on the average indirect effect (AIE) of each head. For GPT-J we found that the increase in performance for many tasks begins to plateau when using $|\mathcal{A}| = 10$ attention heads, though for some tasks using more heads increases performance even more (Figure 6).

The AIE is computed over a subset of all abstractive tasks (Appendix E), using $|\tilde{P}_t| = 25$ corrupted 10-shot prompts per task. Because we are interested in tasks the model can successfully do via ICL, a task is only included if its 10-shot ICL performance is better than the baseline (majority-label) performance. For GPT-J, there are $|\mathcal{T}| = 18$ tasks satisfying this criteria which we use to compute the AIE (Figure 8).

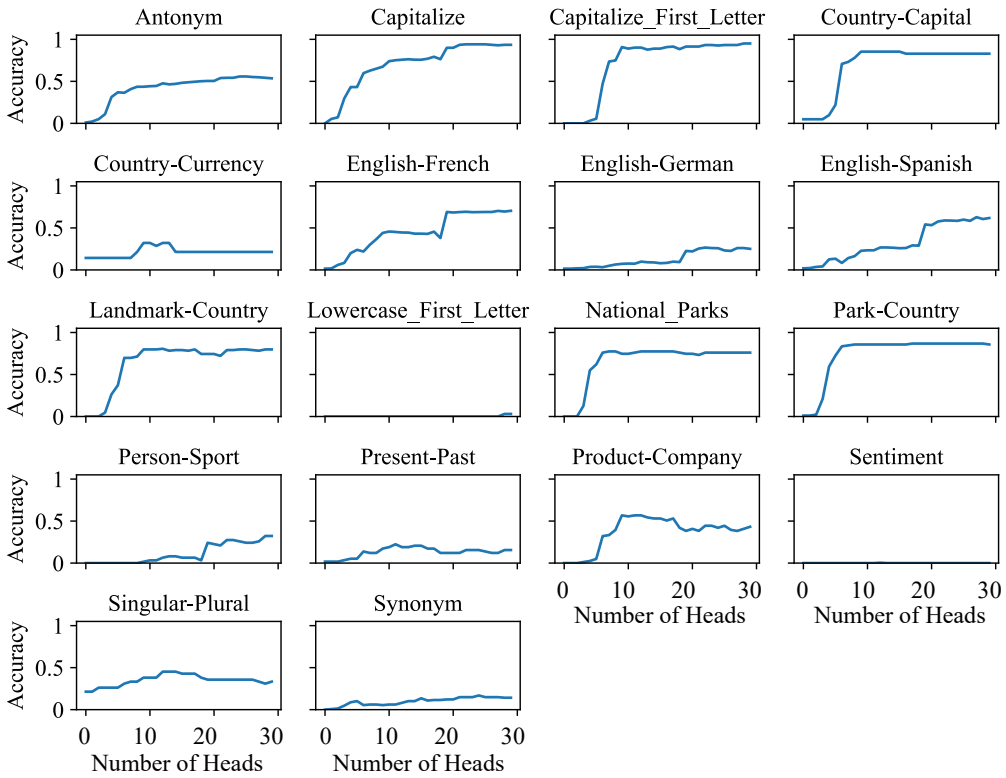

Figure 6: Zero-shot accuracy across 18 different tasks for adding a function vector to GPT-J. We vary the number of heads in $\mathcal{A}$ that are used to create the function vector and find that the change in performance begins to plateau around $|\mathcal{A}| = 10$ attention heads for a majority of the tasks. For this reason, we use $|\mathcal{A}| = 10$ for GPT-J.

Evaluating Function Vectors. To evaluate a function vector's (FV) causal effect, we add the FV to the output of a particular layer in the network ($\mathbf{h}_\ell$) at the last token of a prompt p with query x_q, and then measure whether the predicted word matches the expected answer y_q. This can be expressed simply as $f(p \mid \mathbf{h}_\ell := \mathbf{h}_\ell + v_t)$ (see section B for a more detailed explanation). We report this top-1

accuracy score over the test set under this intervention. If y_q is tokenized as multiple tokens, we use the first token of y_q as the target token.

For all results in Section 3 and in the following appendix sections (unless stated otherwise - e.g. Figure 4), we add the FV to the hidden state at layer $\ell \approx |L|/3$, which we found works well in practice. This corresponds to layer 9 for GPT-J, layer 15 for GPT-NeoX, layer 11 for Llama 2 (7B), layer 14 for Llama 2 (13B) and layer 26 for Llama 2 (70B).

Prompt Templates. The default template we use to construct ICL prompts is: `Q:`$\{x_{ik}\}$`\nA:`$\{y_{ik}\}$`\n\n`, where x_{ik} and y_{ik} (or $\tilde{y}_{ik}$ for corrupted prompts) are substituted for the corresponding element in brackets, and each example is concatenated together. An example of a full prompt template is:

$$\texttt{Q:}\{x_{i1}\}\texttt{\textbackslash nA:}\{y_{i1}\}\texttt{\textbackslash n\textbackslash n} \ \ldots \ \texttt{Q:}\{x_{iN}\}\texttt{\textbackslash nA:}\{y_{iN}\}\texttt{\textbackslash n\textbackslash nQ:}\{x_{iq}\}\texttt{\textbackslash nA:} \tag{15}$$

To evaluate a function vector we use a few different prompt contexts. The shuffled-label prompts are corrupted 10-shot prompts with the same form as (15), while zero-shot prompts only contain a query x_q, without prepended examples (e.g. `Q:`$\{x_{iq}\}$`\nA:`).

In section 3.1 we use FVs extracted from prompts made with the template shown in (15), and test them across a variety of other templates (Table 8).

Table 8: We test a variety of ICL prompt templates in §3.1, which are shown below. The function vectors (FVs) we collect are constructed from a default template of the form `Q:`$\{x_{ik}\}$`\nA:`$\{y_{ik}\}$`\n\n`, and tested on prompts created with the new prompt form.

Template Forms	
`question:`$\{x_{ik}\}$`\n answer:`$\{y_{ik}\}$`\n\n`,	`question:`$\{x_{ik}\}$`\n answer:`$\{y_{ik}\}$`|`,
`A:`$\{x_{ik}\}$`\n B:`$\{y_{ik}\}$`\n\n`,	`Question:`$\{x_{ik}\}$`\n\n Answer:`$\{y_{ik}\}$`\n\n`,
`Input:`$\{x_{ik}\}$` Output:`$\{y_{ik}\}$`|`,	$\{x_{ik}\}$`\n →`$\{y_{ik}\}$`\n\n`,
$\{x_{ik}\}$`\n :`$\{y_{ik}\}$`\n ,`	`input:`$\{x_{ik}\}$` output:`$\{y_{ik}\}$`,`
`question:`$\{x_{ik}\}$` answer:`$\{y_{ik}\}$`,`	`x:`$\{x_{ik}\}$`\n y:`$\{y_{ik}\}$`|`,
`input:`$\{x_{ik}\}$`|output:`$\{y_{ik}\}$`\n`,	$\{x_{ik}\}$` →`$\{y_{ik}\}$`\n\n`,
`input:`$\{x_{ik}\}$` output:`$\{y_{ik}\}$`\n`,	`In:`$\{x_{ik}\}$` Out:`$\{y_{ik}\}$`|`,
`text:`$\{x_{ik}\}$`|label:`$\{y_{ik}\}$`\n\n`,	`x:`$\{x_{ik}\}$` f(x):`$\{y_{ik}\}$`\n`,
`x:`$\{x_{ik}\}$`|y:`$\{y_{ik}\}$`\n\n`,	`A:`$\{x_{ik}\}$` B:`$\{y_{ik}\}$`,`
`text:`$\{x_{ik}\}$`|label:`$\{y_{ik}\}$`\n`,	`x:`$\{x_{ik}\}$`\n y:`$\{y_{ik}\}$`\n\n`

D RESULTS INCLUDING INCORRECT ICL

For simplicity of presentation in Section 3, we filter the test set to cases where the model correctly predicts y_q given a 10-shot ICL prompt containing query x_q. In this section we compare those results to the setting in which correct-answer filtering is *not* applied. When filtering is not applied, the causal effects of function vectors remain essentially unchanged (Figure 7).

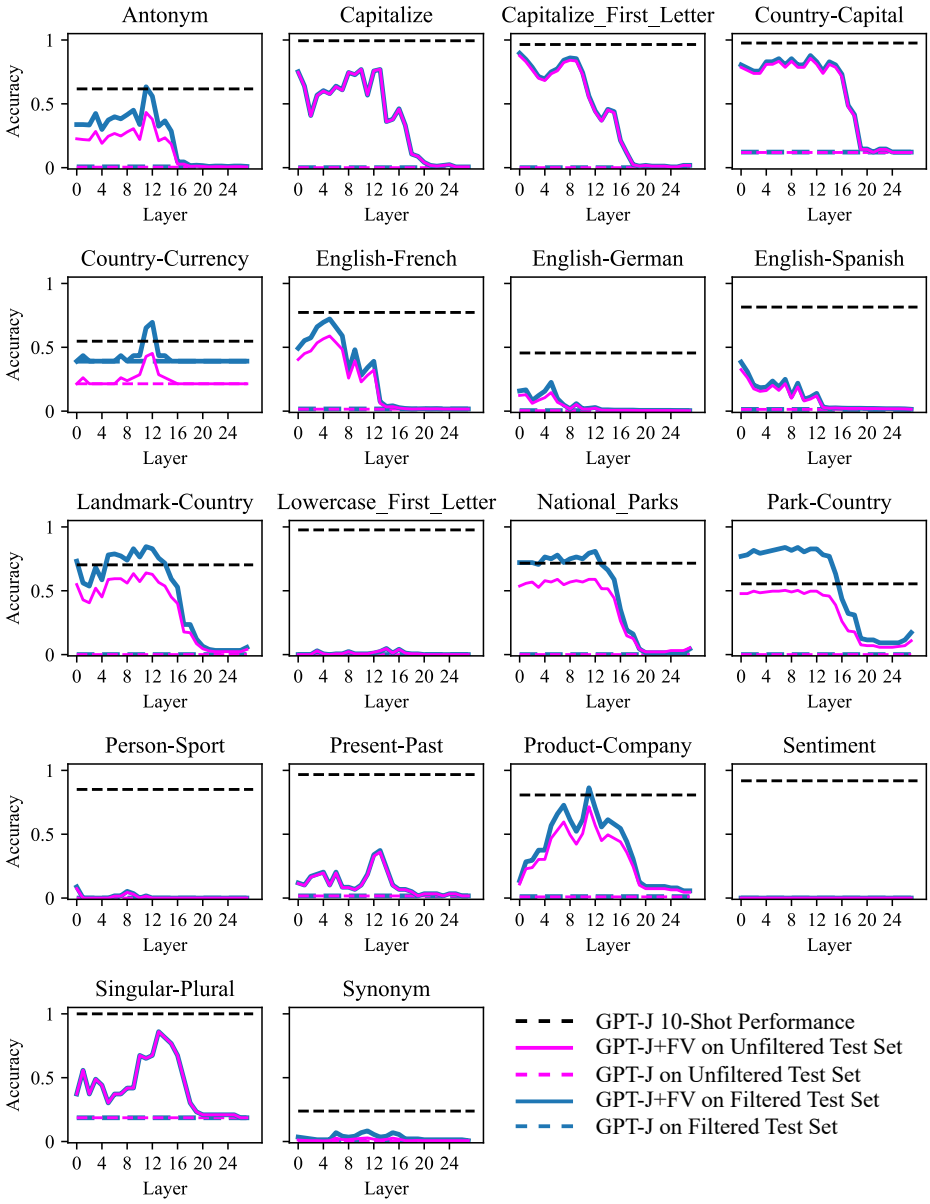

Figure 7: Comparing layer-wise zero-shot results of adding a function vector to GPT-J with and without filtering the task test set to cases where GPT-J correctly answers a 10-shot prompt. The results when filtering the test set in this manner are shown in blue, while the results without filtering the test set are shown in magenta. In both cases the performance is very similar, with performance dropping only slightly on a few tasks when not filtering the dataset to correct answers. The black dashed line corresponds to the oracle accuracy (GPT-J's 10-shot performance) in the unfiltered setting, while colored dashed lines correspond to GPT-J's performance in the zero-shot setting.